UNIVERSIDADE KIMPA VITA
ESCOLA SUPERIOR POLITÉCNICO DO UÍGE
CURSO DE ENGENHARIA INFORMÁTICA

Trabalho em grupo de Linguagem de Programação VI

Classe: 3º
Turma: Única
Período: Manhã
Data: 18 abril
Orientado: Moyo Kanivengidio

MEMBROS:
1-António Lopes Tamba
2-Arnano Quiesse Muanza Mavinga
3-Augusto Paulo
4-Clemente Manuel Dinis José
5-Lukombo David
6-Jeremias Manuel
7-João Nkembo
8-Maria David Kiala
9-Nsumbo António Ngombo
10-Rosa Sozinho Garcia

TEMA:

Árvores e Complexidade de Algoritmo de tempo

Objetivo:

- Comprender o conteudo de Arvore e de Complexidade

O que é uma Árvore?

Uma Árvore é uma estrutura de dados hierárquica e não linear, composta por nós (nodes) e arestas (edges). Possui um nó raiz (root) do qual todos os outros nós descendem, formando relações pai-filho.

Sistemas de Ficheiros

Pastas e subpastas organizadas em hierarquia

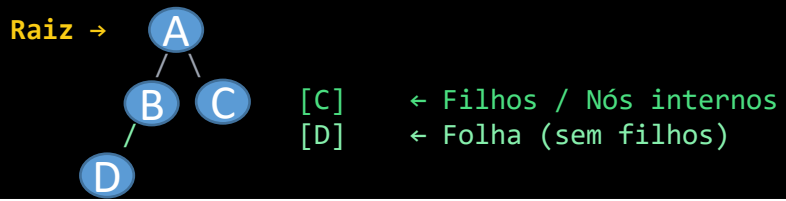
Busca Eficiente

BST permite busca em O(log n)

DOM / HTML

Browsers usam árvores para renderizar páginas

Representação Visual:



Terminologia Básica

Raiz (Root)

Nó superior da árvore — o ponto de entrada. Não tem pai.

Nó (Node)

Elemento que armazena um valor e referências aos filhos.

Aresta (Edge)

Ligação/conexão entre dois nós (pai → filho).

Folha (Leaf)

Nó sem filhos — extremidade da árvore.

Subárvore

Qualquer nó + seus descendentes formam uma subárvore.

Altura (Height)

Nº de arestas do caminho mais longo da raiz até uma folha.

Profundidade

Distância (em arestas) de um nó até a raiz.

Grau (Degree)

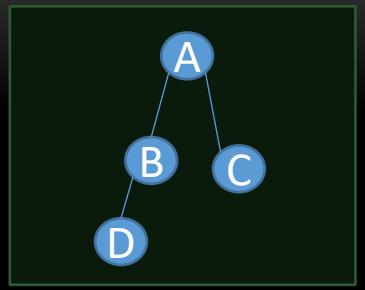
Número de filhos de um nó. Folhas têm grau 0.

Altura de uma árvore balanceada com N nós: O(log N)

Tipos de Árvores

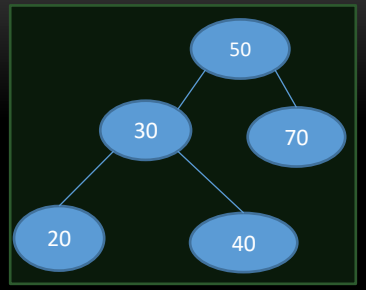
Árvore Binária

Cada nó tem no máximo 2 filhos (esquerdo e direito).



BST (Binary Search Tree)

Variante binária com regra: esquerda < nó < direita. Permite busca eficiente em O(log n).



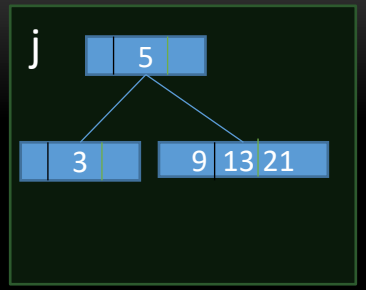
Árvore AVL

BST auto-balanceada. O fator de balanceamento de cada nó é -1, 0 ou +1. Garante O(log n) sempre.

```
Balanceada
automaticamente
FB = altura(D) -
altura(E) ∈ {-1,0,1}
```

Árvore B / B+

Generalização com múltiplos filhos por nó. Usada em bancos de dados e sistemas de ficheiros (NTFS, ext4).



Existem também: Tries, Segment Trees, Árvores Rubro-Negras, Heaps...

BST — Implementação em Python

```python
class NoArvore:
  def __init__(self, valor):
    self.valor = valor
    self.esquerda = None
    self.direita  = None


class
ArvoreBuscaBinaria:
  def inserir(self, valor):
    # Insere na posição correta
    ...


  def buscar(self, valor):
    # Retorna True/False
    ...


  def em_ordem(self):
    # Retorna lista ordenada
    ...
```

A BST garante que em_ordem() sempre retorna os elementos em ordem crescente

Regra da BST

```
esquerda < nó
direita   > nó
duplicados ignorados
```

Exemplo de uso:

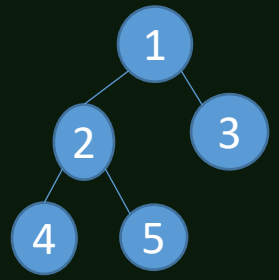
```
ab.inserir(50)
ab.inserir(30)
ab.inserir(70)

em_ordem() →
[20,30,40,50,60,70,80]

buscar(40) → True
buscar(99) → False
```

Travessias (Tree Traversals)

Árvore de exemplo:



In-Order (E → R → D)

Resultado: 4 → 2 → 5 → 1 → 3

Visita: Esquerda → Raiz → Direita
Na BST, gera elementos em ordem crescente! ✅

```
def em_ordem(no):
  if no:
    em_ordem(no.esq)
    print(no.val)
    em_ordem(no.dir)
```

Pre-Order (R → E → D)

Resultado: 1 → 2 → 4 → 5 → 3

Visita: Raiz → Esquerda → Direita
Usado para copiar/serializar árvores.

```
def pre_ordem(no):
  if no:
    print(no.val)
    pre_ordem(no.esq)
    pre_ordem(no.dir)
```

Post-Order (E → D → R)

Resultado: 4 → 5 → 2 → 3 → 1

Visita: Esquerda → Direita → Raiz
Usado para deletar árvore ou avaliar expressões.

```
def pos_ordem(no):
  if no:
    pos_ordem(no.esq)
    pos_ordem(no.dir)
    print(no.val)
```

Existe também o BFS (Level-Order): visita nó a nó por nível, usando uma fila (Queue)

Complexidade de Tempo

Operação	BST (Melhor)	BST (Médio)	BST (Pior)	AVL / Bal.
Busca (Search)	O(log n)	O(log n)	O(n)	O(log n)
Inserção	O(log n)	O(log n)	O(n)	O(log n)
Remoção	O(log n)	O(log n)	O(n)	O(log n)
Mín / Máx	O(log n)	O(log n)	O(n)	O(log n)
Travessia	O(n)	O(n)	O(n)	O(n)
Espaço (Space)	O(n)	O(n)	O(n)	O(n)

Pior caso O(n) ocorre quando a BST fica degenerada (tipo lista ligada) — inserir elementos já ordenados! Solução: AVL ou Red-Black Tree.

O(log n) = eficiente | O(n) = degenerado (evitar!)

Aplicações no Mundo Real

Bancos de Dados

MySQL, PostgreSQL e SQLite usam Árvores B+ para indexar tabelas. Permite encontrar registos em O(log n) mesmo com milhões de linhas.

DOM do Browser

HTML é uma árvore. O browser percorre-a para renderizar páginas. Frameworks como React usam uma Virtual DOM em árvore.

Sistema de Ficheiros

NTFS (Windows) e ext4 (Linux) organizam pastas e ficheiros em árvores B. Operações de busca são extremamente rápidas.

IA & Jogos

Árvore de decisão (Decision Tree) em ML. Minimax em xadrez/damas usa árvores de jogo para avaliar jogadas futuras.

Autocomplete / DNS

Trie (Prefix Tree) permite autocompletar palavras eficientemente. DNS usa árvores hierárquicas para resolver nomes.

Heaps & Filas

Heap (min/max) é uma árvore binária usada em filas de prioridade, algoritmo de Dijkstra e HeapSort (O(n log n)).

Árvores estão em todo lugar — se não vês, é porque estão bem escondidas numa abstração

Resumo

Neste pequeno capítulo aprendemos:

Árvores

Estrutura hierárquica com raiz, nós, folhas e arestas

Terminologia

Root, Node, Edge, Leaf, Height, Degree, Subtree

BST

Esquerda < Nó < Direita → busca eficiente O(log n)

Travessias

In-Order (ordenado), Pre-Order (cópia), Post-Order (delete)

Complexidade

O(log n) médio · O(n) pior caso (degenerada)

Tipos

Binária, BST, AVL, B/B+, Trie, Heap, Red-Black...

"Uma árvore sem raiz não é nada — assim como um programa sem estrutura de dados "

COMPLEXIDADE DE ALGORITMO DE TEMPO

O que é Complexidade de Tempo?

É uma forma de medir como o tempo de execução de um algoritmo cresce em relação ao tamanho da entrada (n).

Não mede segundos — mede operações!

Melhor Caso

Ω (Omega)
Cenário mais favorável

Caso Médio

Θ (Theta)
Comportamento típico

Pior Caso

O (Big-O)
Garantia máxima

Na prática usamos quase sempre Big-O (pior caso) — melhor preparar para o apocalipse! 😅

As Notações Big-O

← MAIS RÁPIDO

MAIS LENTO →

Constante	Logarítmica	Linear	Linearítmica	Quadrática	Exponencial
Aceder array[0]	*Busca binária*	*Loop simples*	*Merge Sort*	*Bubble Sort (pior)*	*Fibonacci recursivo*

Na Prática: Código & Complexidade

O(1) — Constante

```
def getFirst(arr):
  return arr[0];
```

Array com 10 ou 10 milhões — sempre 1 operação!

O(n^2) — Quadrática

```
for (let i=0; i<n; i++):
  for (let j=0; j<n; j++) :
    // loop dentro de loop
```

n=1000 → 1.000.000 ops. n=10000 → 100.000.000 🤯

Regra de ouro

Loops aninhados = multiplicar complexidades.

1 loop → O(n) • 2 loops aninhados → O(n^2) • 3 loops → O(n^3) (hora de repensar a vida)

Crescimento com n = 1.000 operações

Notação	Operações (n=1000)	Velocidade	Avaliação
O(1)	1	⚡ ⚡ ⚡ ⚡ ⚡	Excelente
O(log n)	~10	⚡ ⚡ ⚡ ⚡	Muito bom
O(n)	1.000	⚡ ⚡ ⚡	Bom
O(n log n)	~10.000	⚡ ⚡	Aceitável
$O(n^2)$	1.000.000	⚡	Evitar
$O(2^n)$	10^301	☠	Impraticável

Como Escrever Código Eficiente

01 Evita loops desnecessários

Antes de aninhar loops, pergunta-te: existe uma estrutura de dados melhor?

02 Usa estruturas adequadas

HashMap/Set para buscas → O(1). Arrays ordenados + Busca Binária → O(log n).

03 Divide e conquista

Algoritmos como Merge Sort dividem o problema → passam de $O(n^2)$ para O(n log n).

04 Memoization / Cache

Guarda resultados já calculados. Fibonacci recursivo $O(2^n)$ → vira O(n) com cache!

Resumo do Capítulo

- Big-O mede crescimento, não velocidade absoluta
- O(1) e O(log n) são os sonhos de qualquer programador
- $O(n^2)$ é tolerável para n pequeno — mas escala mal
- $O(2^n)$ na produção = pedido de demissão automático
- Sempre analisa antes de optimizar — sem métricas, é chute

ALGORITMOS DE ORDENAÇÃO

Bubble, Merge & Quick Sort

Três formas de ordenar — uma é claramente melhor que as outras. Spoiler: não é o Bubble

Bubble Sort

$O(n^2)$

O mais simples.
O mais lento.
O favorito das aulas.

Merge Sort

$O(n \log n)$

Divide e conquista.
Estável e previsível.
O académico.

Quick Sort

$O(n \log n)^*$

Rápido na prática.
Astuto mas traiçoeiro.
O guerreiro.

Bubble Sort

$O(n^2)$

Como funciona?

Compara elementos vizinhos e troca os que estao fora de ordem.

Repete ate o array estar ordenado — como bolhas a subir.

Exemplo visual: [5, 3, 8, 1]

```
[5, 3, 8, 1]  →  compara 5,3

[3, 5, 8, 1]  →  troca! compara 5,8

[3, 5, 8, 1]  →  ok. compara 8,1

[3, 5, 1, 8]  →  troca! repete...
```

Código

```python
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1],
arr[j]
    return arr

print(bubble_sort([5, 3, 8, 1]))  # [1, 3, 5, 8]
```

Melhor: $O(n)$ | Medio: $O(n^2)$ | Pior: $O(n^2)$ | Espaco: $O(1)$ | Estavel: Sim | Uso real: Quase nenhum (so para aprender 😅)

Merge Sort

O(n log n)

Como funciona?

Divide o array ao meio recursivamente ate ter arrays de 1 elemento.

Depois junta (merge) os pedacos em ordem. Divide e conquista!

Divisao: [8, 3, 5, 1]

```
[8, 3, 5, 1]

[8, 3]      [5, 1]

[8] [3]   [5] [1]

[3, 8]    [1, 5]  →  merge

[1, 3, 5, 8]  resultado!
```

Código

```python
def merge_sort(arr):
    if len(arr) <= 1:
        return arr
    mid = len(arr) // 2
    left = merge_sort(arr[:mid])
    right = merge_sort(arr[mid:])
    return merge(left, right)

def merge(l, r):
    res, i, j = [], 0, 0
    while i < len(l) and j < len(r):
        if l[i] < r[j]:
            res.append(l[i]); i += 1
        else:
            res.append(r[j]); j += 1
    return res + l[i:] + r[j:]

print(merge_sort([8, 3, 5, 1]))  # [1, 3, 5, 8]
```

Melhor: O(n log n) | Medio: O(n log n) | Pior: O(n log n) | Espaco: O(n) | Estavel: Sim | Uso real: Alto

Quick Sort

O(n log n)*

Como funciona?

Escolhe um elemento pivô. Coloca menores à esquerda, maiores à direita.

Repete recursivamente em cada partição. Sem array extra!

Exemplo com pivot=5: [3, 8, 5, 1, 9]

```
pivot = 5

[3, 1]  [5]  [8, 9]

ordena [3,1] e [8,9]...

[1, 3, 5, 8, 9]  resultado!
```

**Pior caso: O(n²) se pivot for sempre o menor/maior!*

código

```
def quick_sort(arr):
    if len(arr) <= 1:
        return arr
    pivot = arr[-1]
    left  = [x for x in arr[:-1] if x <= pivot]
    right = [x for x in arr[:-1] if x > pivot]
    return quick_sort(left) + [pivot] +
quick_sort(right)

print(quick_sort([3, 8, 5, 1, 9]))  # [1, 3, 5, 8,
9]
```

Melhor: O(n log n) | Medio: O(n log n) | Pior: O(n²) | Espaco: O(log n) | Estavel: Nao | Uso real: Muito alto

Comparacao Final: Qual Usar?

Algoritmo	Melhor	Medio	Pior	Espaco	Estavel
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$	$O(1)$	Sim
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$	$O(n)$	Sim
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$	$O(\log n)$	Nao

Veredicto

Pequeno dataset ou fins didaticos: **Bubble Sort** (so para aprender!)
Precisa de estabilidade garantida: **Merge Sort** (previsivel e seguro)
Performance maxima na pratica: **Quick Sort** (favorito dos sistemas reais)

Referências

Livros

Introduction to Algorithms (CLRS) — Thomas Cormen et al.

Grokking Algorithms — Aditya Bhargava.

Sites gratuitos

visualgo.net - visualiza Bubble, Merge, Quick

bigocheatsheet.com - tabela completa de complexidades de todos os algoritmos clássicos

geeksforgeeks.org - artigos detalhados com código em Python para cada algoritmo